Analyse

SAÉ 3.02 - Labyrinthe

Solution aux besoins

Étudiants :

Bredelle Victor
Marouze Antonin
Harlaut Romain
Zheng Angèl
Lavogiez Baptiste

Enseignant :

Mme Boneva

16 octobre 2025

Table des matières

1 Introduction **2**

2 Équipe et organisation du travail **3**

3 Cas d'utilisation **5**

3.1 Diagramme de cas d'utilisation 5

3.2 Analyse du diagramme de cas d'utilisation 5

3.2.1 Acteurs identifiés 5

3.2.2 Description des cas d'utilisation 6

3.2.3 Clarifications 6

3.3 Fiches descriptives 7

3.3.1 Cas d'utilisation : Lancer mode progression 7

3.3.2 Cas d'utilisation : Charger profil 8

3.3.3 Cas d'utilisation : Afficher progression 9

3.3.4 Cas d'utilisation : Déplacer joueur 10

3.4 Prototypes d'interface 11

3.4.1 Vue d'ensemble et parcours utilisateur 11

3.4.2 Correspondance avec les cas d'utilisation 12

3.4.3 Clarifications 12

4 Diagramme de classe **14**

4.1 Structure générale 14

4.2 Conformité avec la méthode vue en cours 15

4.3 Correspondance avec les cas d'utilisation 15

4.4 Clarifications 15

5 Conclusion **18**

1 Introduction

Ce rapport présente l'analyse UML du projet **SAÉ S3.02 - Labyrinthe**, un jeu avec deux modes (libre et progression) :

1. Diagrammes de cas d'utilisation
2. Fiches descriptives
3. Prototype d'interface
4. Diagramme de classe

2 Équipe et organisation du travail

Groupe G3.3

- Bredelle Victor
- Marouze Antonin
- Harlaut Romain
- Zheng Angèl
- Lavogiez Baptiste

> **1. Victor**

Rédaction de trois fiches descriptives avec Baptiste : Charger profil, Afficher progression, Lancer mode progression.

> **2. Antonin**

Réalisation de la maquette *Figma* avec Angèl et Romain.

> **3. Romain**

Réalisation de la fiche descriptive "Déplacer joueur" et aide à la maquette *Figma*.

> **4. Angèl**

Réalisation de la maquette *Figma* avec Antonin et Romain.

5. Baptiste

Rédaction de trois fiches descriptives avec Victor et rédaction du rapport.

Le groupe

Information

L'ensemble des contributeurs a collaboré à l'élaboration du diagramme de classes.

3 Cas d'utilisation

3.1 Diagramme de cas d'utilisation

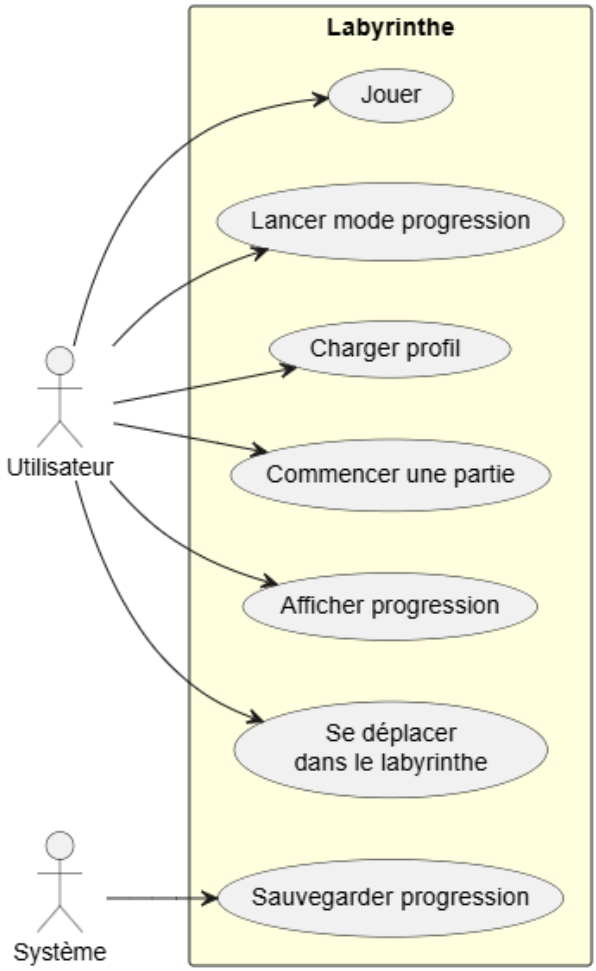

Figure 3.1 – Cas d'utilisation identifiés

3.2 Analyse du diagramme de cas d'utilisation

Le diagramme identifie **sept cas d'utilisation** répartis entre deux acteurs.

3.2.1 Acteurs identifiés

- **Utilisateur** : Acteur principal représentant le joueur.
- **Système** : Acteur secondaire représentant les processus de persistance automatique des données.

L'acteur "Système" modélise la sauvegarde automatique post-action.

3.2.2 Description des cas d'utilisation

Le diagramme couvre le **mode progression** :

1. **Jouer** : Point d'entrée du jeu.
2. **Lancer en mode progression** : Initialisation du mode progression (saisie du nom).
3. **Charger profil** : Reprise d'une progression existante.
4. **Commencer une partie** : Démarrage d'un labyrinthe (défi ou configuration libre).
5. **Afficher la progression** : Consultation du tableau de bord du joueur.
6. **Se déplacer dans le labyrinthe** : Déplacement case par case.
7. **Sauvegarder la progression** : Persistance automatique de l'état du joueur.

3.2.3 Clarifications

Choix de modélisation :

- **"Jouer"** est un cas vu en TD. Il concerne un bouton "Jouer" qui permet d'accéder au choix du mode de jeu.
- **"Charger profil"** spécialise "Lancer mode progression" si le nom existe déjà.
- **Sauvegarde automatique** : Modélisée par l'acteur "Système" pour distinguer l'automatisme des actions explicites.
- **Mode libre** : Réutilise "Jouer" et "Commencer une partie" (avec configuration manuelle : largeur, hauteur, pourcentage de murs) mais sans authentification, profil ni sauvegarde. Le joueur accède directement au labyrinthe après configuration.

3.3 Fiches descriptives

3.3.1 Cas d'utilisation : Lancer mode progression

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun
Préconditions : Aucun
Garanties en cas de succès : L'utilisateur accède au mode progression.
Garanties minimales : L'intégrité de la base de données est préservée.
Scénario nominal :

1. L'utilisateur sélectionne la fonctionnalité "Lancer mode progression".
2. Le système affiche un menu où l'utilisateur peut saisir son nom.
3. L'utilisateur saisit son nom.
4. Le système vérifie que le nom est valide, sauvegarde son profil dans la base de données et lance le mode progression.

Scénarios alternatifs :

A : Le système détecte une erreur lors de la vérification
4(A) : Le système affiche une notification d'erreur, puis revient à l'étape 2 du scénario nominal.

B : Le système détecte une occurrence du nom dans la base de données (charger profil)
4(B) : Le système démarre le **Cas d'utilisation** "Charger profil".

3.3.2 Cas d'utilisation : Charger profil

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Le système détecte une occurrence du nom saisi par l'utilisateur dans la base de données.
Préconditions : L'utilisateur est dans le menu de saisie du nom et a validé sa saisie.
Garanties en cas de succès : Le système charge le profil associé au nom et l'utilisateur reprend sa progression.
Garanties minimales : L'intégrité des données existantes est préservée.
Scénario nominal :

1. Le système affiche un menu où l'utilisateur peut choisir entre "Nouveau profil" ou "Reprendre".
2. L'utilisateur sélectionne "Reprendre".
3. Le système charge le profil associé au nom et l'utilisateur reprend sa progression.

Scénarios alternatifs :

A : L'utilisateur choisit "Nouveau profil"
2(A) : Le système affiche une demande de confirmation à l'utilisateur (action sensible).
3(A) : L'utilisateur confirme.
4(A) : Le système remplace les données du profil existant et crée un nouveau profil vide. Puis, l'utilisateur accède alors au mode progression.

B : L'utilisateur annule la confirmation
3(B) : Le système revient en étape 1 du scénario nominal.

Information

Le profil existant n'est pas supprimé : il est renommé automatiquement, par exemple, en "Old+compte" afin d'en conserver une sauvegarde en prévision de toute erreur. À titre illustratif : `toto` est converti en `totoOld1`.

Clarification

Ce cas est la suite du scénario alternatif B de "Lancer mode progression".

3.3.3 Cas d'utilisation : Afficher progression

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun
Préconditions : L'utilisateur possède un profil et se trouve en mode progression.
Garanties en cas de succès : Le système affiche la progression associée au joueur.
Garanties minimales : Les données affichées sont cohérentes et non modifiées.
Scénario nominal :

1. L'utilisateur sélectionne la fonctionnalité "accéder au menu du mode progression".
2. Le système calcule le score et les caractéristiques du joueur, puis affiche sa progression.

Scénarios alternatifs : Aucun

3.3.4 Cas d'utilisation : Déplacer joueur

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun
Préconditions : Le joueur se trouve dans le labyrinthe.
Garanties en cas de succès : Le système déplace le joueur à la case souhaitée.
Garanties minimales : La position du joueur reste cohérente.
Scénario nominal :

1. Le joueur appuie sur une flèche directionnelle.
2. Le système vérifie que la direction est accessible et déplace le joueur sur la case associée.

Scénarios alternatifs :

A : Le joueur atteint la sortie
2(A) : Le système notifie la réussite, sauvegarde la progression et redirige vers le menu de sélection de défi.

B : Le joueur se déplace dans une case inaccessible
2(B) : Le système ne modifie pas la position du joueur et le scénario reprend à l'étape 1.

3.4 Prototypes d'interface

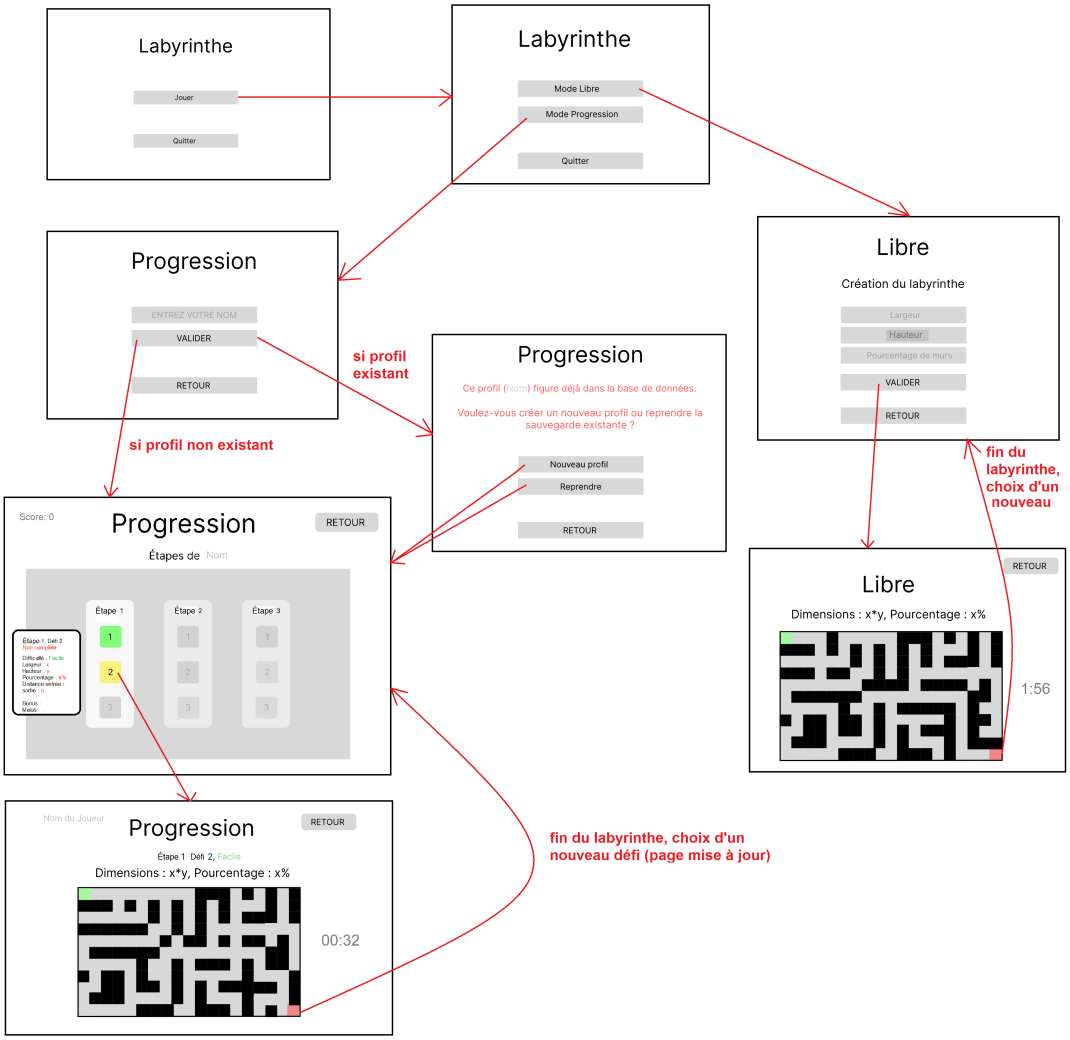

Figure 3.2 – Prototypes d'interface identifiés avec arborescence

3.4.1 Vue d'ensemble et parcours utilisateur

Le prototype couvre **huit écrans** répartis entre les deux modes de jeu.

Parcours utilisateur – Mode Progression (6 écrans) :

1. **Menu d'accueil** : Options "Jouer" et "Quitter".
2. **Choix du mode** : Sélection "Mode Libre" ou "Mode Progression".
3. **Saisie du nom** : Identification ou création du profil.
4. **Gestion du profil existant** : Choix "Nouveau profil" ou "Reprendre".

5. **Sélection étape/défi** : Tableau de progression avec code couleur.
6. **Interface de jeu** : Labyrinthe avec joueur et sortie.

Parcours utilisateur – Mode Libre (4 écrans) :

1. **Menu d'accueil**
2. **Choix du mode**
3. **Configuration du labyrinthe** : Paramétrage des dimensions et murs.
4. **Interface de jeu** : Labyrinthe généré aléatoirement.

3.4.2 Correspondance avec les cas d'utilisation

Le tableau suivant établit la correspondance entre les écrans du prototype (mode progression) et les cas d'utilisation.

Cas d'utilisation	**Écrans concernés**
Lancer mode progression	Choix du mode → Saisie du nom
Charger profil	Page de profil existant (choix "Reprendre" ou "Nouvelle partie")
Jouer	Menu d'accueil → Clic sur jouer
Commencer une partie	Sélection étape/défi (clic sur un défi) → Labyrinthe en jeu
Afficher progression	Sélection étape/défi (affichage et survol des défis)
Se déplacer dans le labyrinthe	Labyrinthe en jeu
Sauvegarder progression	Automatique après sortie du labyrinthe

Cette traçabilité assure la correspondance entre fonctionnalités et interface.

3.4.3 Clarifications

Boutons "Retour"/"Quitter" omis par clarté.

L'écran de progression s'adapte au profil chargé.

Cet écran intègre visualisation et sélection des défis. Le survol affiche les détails, le clic lance le défi.

Détail :

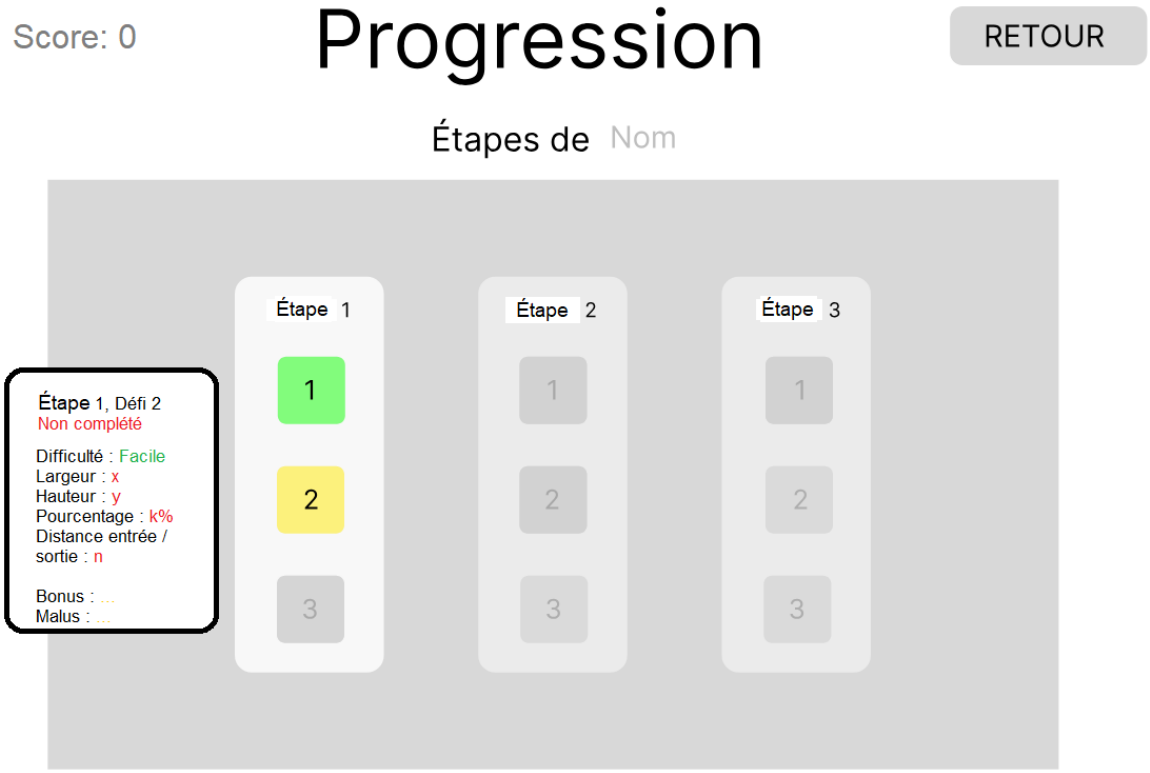

Figure 3.3 – Page de progression avec survol d'un défi

À la sortie du labyrinthe, sauvegarde et mise à jour automatiques.

4 Diagramme de classe

Figure 4.1 – Diagramme de classe identifié

4.1 Structure générale

L'architecture se décompose en trois niveaux conceptuels :

1. **Niveau contrôleur** : `Main` orchestre le flux général.
2. **Niveau logique métier** : `GameMode`, `Maze`, `Player`, etc. encapsulent les règles métier.
3. **Niveau persistance** : `PlayerDatabase` isole les opérations de sauvegarde/chargement.

Cette séparation facilite la maintenance et la testabilité.

4.2 Conformité avec la méthode vue en cours

La méthodologie préconise une classe `Main` avec une fonction par cas d'utilisation, structure respectée ici.

4.3 Correspondance avec les cas d'utilisation

La correspondance suivante démontre que la classe `Main` couvre l'ensemble des cas d'utilisation.

Fonctions du `Main`	**Cas d'utilisation correspondants**
`play()`	Jouer
`startGame(): GameMode`	Lancer mode progression
`movePlayerPosition(...)`	Se déplacer dans le labyrinthe
`displayProgress(player: Player)`	Afficher progression
`savePlayer(player: Player)`	Sauvegarder progression
`loadPlayer(name: String): Player`	Charger profil
`createMaze(...)`	Commencer une partie

Le type abstrait `GameMode` mutualise le code commun aux deux modes.

4.4 Clarifications

> **Génération aléatoire**

La génération aléatoire n'est pas concernée par ce rapport car l'algorithme interne n'est pas demandé dans une analyse UML.

Les étapes et défis

Information

Un `Level` correspond à une *Étape* (pour éviter le conflit avec `Stage` de JavaFX).
Un `Challenge` est un défi.

Il y a 3 étapes et 3 défis par étape, les structures de données ont donc une taille fixe.

Un objet `Challenge` comporte les attributs requis par le sujet.

La méthode `isCompleted(): boolean` de `Level` retourne `True` si au moins un défi est complété.

Les méthodes d'accès direct aux étapes (getters/setters) ne sont pas mentionnées car non demandées dans le plan.

Le type `GameMode`

`startGame()` retourne un `GameMode` (`FreeMode` ou `ProgressionMode`). Ce type abstrait unifie les opérations communes et évite la duplication.

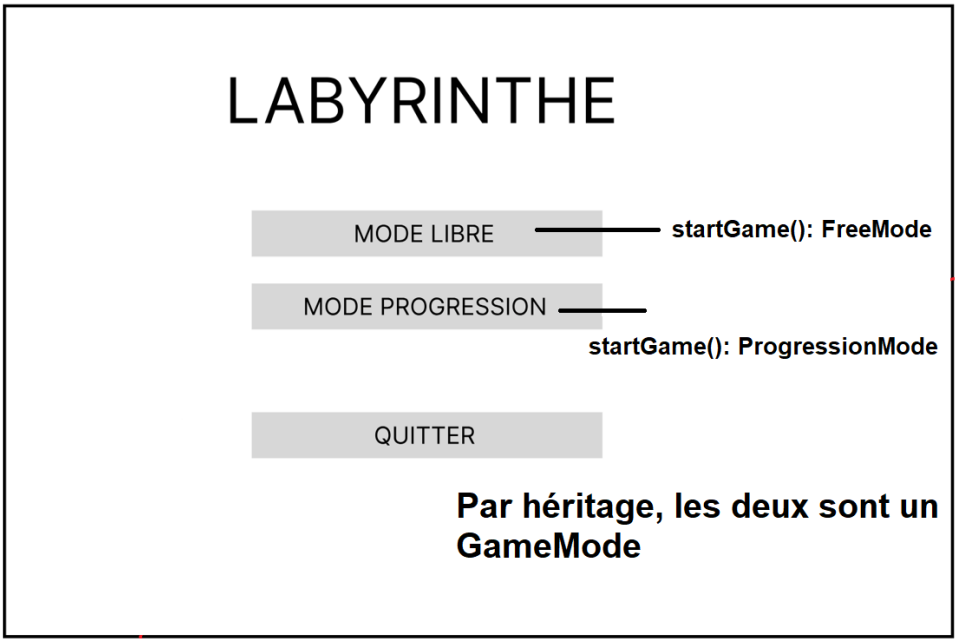

Figure 4.2 – startGame() dans le Menu

L'initialisation de `Player` dans `ProgressionMode`

Le constructeur de `ProgressionMode` invoque `loadPlayer(name: String): Player` pour :

- Charger un profil existant si le nom est trouvé.
- Créer un nouveau profil sinon.

Le `Player` devient attribut de `ProgressionMode`, garantissant l'exclusivité de gestion des profils.

5 Conclusion

Cette analyse répond rigoureusement aux exigences du jalon 1, posant ainsi des fondations robustes pour l'implémentation.